

UNIVERSIDADE ESTADUAL PAULISTA "JÚLIO DE MESQUITA FILHO"
FACULDADE DE CIÊNCIAS - CAMPUS BAURU
DEPARTAMENTO DE COMPUTAÇÃO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

GUSTAVO YUJII SILVA KADOOKA

**EXPLORANDO ALGORITMOS DE COMPRESSÃO DE DADOS:
TEORIA, IMPLEMENTAÇÃO E DESEMPENHO**

BAURU
Novembro/2025

GUSTAVO YUJII SILVA KADOOKA

**EXPLORANDO ALGORITMOS DE COMPRESSÃO DE DADOS:
TEORIA, IMPLEMENTAÇÃO E DESEMPENHO**

Trabalho de Conclusão de Curso em
Ciência da Computação apresentado
ao Departamento de Computação da
Universidade Estadual Paulista “Júlio de
Mesquita Filho”, Faculdade de Ciências,
Campus Bauru.

Orientador: Profa. Dra. Andréa Carla
Gonçalves Vianna

K11e

Kadooka, Gustavo Yujii Silva

Explorando Algoritmos de Compressão de Dados: Teoria,
Implementação e Desempenho / Gustavo Yujii Silva Kadooka. --
Bauru, 2025

60 p. : il., tabs.

Trabalho de conclusão de curso (Bacharelado - Ciência da
Computação) - Universidade Estadual Paulista (UNESP), Faculdade
de Ciências, Bauru

Orientadora: Andréa Carla Gonçalves Vianna

1. Compressão de dados. 2. Teoria da informação. 3. Compressão
sem perdas – Huffman, LZ77, LZW, GZIP. I. Título.

Gustavo Yujii Silva Kadooka

Explorando Algoritmos de Compressão de Dados: Teoria, Implementação e Desempenho

Trabalho de Conclusão de Curso em Ciência da Computação apresentado ao Departamento de Computação da Universidade Estadual Paulista “Júlio de Mesquita Filho”, Faculdade de Ciências, Campus Bauru.

Banca Examinadora

**Profa. Dra. Andréa Carla Gonçalves
Vianna**

Orientador

Universidade Estadual Paulista “Júlio de
Mesquita Filho”

Faculdade de Ciências

Departamento de Ciência da Computação

Professor Convidado 1

Universidade Estadual Paulista “Júlio de
Mesquita Filho”

Faculdade de Ciências

Departamento de Ciência da Computação

Professor Convidado 2

Universidade Estadual Paulista “Júlio de
Mesquita Filho”

Faculdade de Ciências

Departamento de Ciência da Computação

Bauru, 11 de novembro de 2025.

À minha amada família, fonte inesgotável de inspiração e força.

Agradecimentos

Agradeço à Universidade Estadual Paulista “Júlio de Mesquita Filho” – Faculdade de Ciências, Campus de Bauru –, por ter sido o espaço onde pude me desenvolver como estudante, profissional e indivíduo. Sou grato a todos os professores e funcionários da instituição pelo conhecimento compartilhado e pelo apoio ao longo da graduação.

À minha orientadora, Profa. Dra. Andréa Carla Gonçalves Vianna, pela disponibilidade, paciência diante das dificuldades e por acreditar no meu potencial durante o desenvolvimento deste trabalho.

Aos meus colegas de curso, que estiveram presentes ao longo desta jornada, dividindo aprendizados, desafios e conquistas. O convívio e a troca de experiências tornaram a caminhada mais leve e significativa.

À minha irmã Gabriely, pelas palavras de incentivo e carinho em todos os momentos, mesmo nos mais difíceis.

E, com o meu mais profundo e sincero agradecimento, menciono especialmente meus pais – Flávia e Silvio –, por todo o apoio emocional, pelos ensinamentos de vida, pela confiança depositada em mim e por sempre me encorajarem a seguir em frente. Esta conquista é, acima de tudo, de vocês.

Resumo

Este trabalho apresenta um estudo abrangente, a implementação prática e a análise de desempenho de algoritmos clássicos de compressão de dados, a saber: Huffman, LZ77, LZW e GZIP. São explorados os conceitos teóricos fundamentais de cada algoritmo, seguidos pela implementação prática utilizando a linguagem de programação C++. As implementações dos algoritmos lidam com os formatos de arquivo de texto simples (.txt), imagens bitmap (.bmp) e arquivos de áudio PCM (.wav), demonstrando versatilidade e aplicabilidade em diferentes domínios de dados. Para facilitar a interação do usuário, foi desenvolvida uma interface gráfica utilizando a biblioteca GTK, permitindo a seleção dos algoritmos de compressão e o arquivo de entrada. Ademais, para realizar uma análise comparativa rigorosa, foram utilizados scripts em Python para processar os dados experimentais e gerar representações gráficas dos principais indicadores de desempenho, como a taxa de compressão e o tempo de execução para diferentes tipos de arquivos. Os resultados experimentais indicam que não há um algoritmo que se destaque universalmente, pois cada um apresenta vantagens específicas conforme o tipo de arquivo e as características dos dados. Isso ressalta a importância da escolha do método de compressão adequado às necessidades específicas da aplicação. O projeto contribui tanto com uma ferramenta prática para compressão de dados quanto com um recurso educacional que auxilia na compreensão dos algoritmos clássicos e seu comportamento em cenários reais.

Palavras-chave: Compressão de dados. Algoritmos de compressão. Huffman. LZ77. LZW. GZIP. C++. Python. GTK.

Abstract

This work presents a comprehensive study, implementation, and performance analysis of classical data compression algorithms, namely Huffman, LZ77, LZW, and GZIP. The theoretical concepts behind each algorithm are explored in depth, followed by practical implementations using C++ programming language. The implementations support plain text (.txt), bitmap images (.bmp), and PCM audio files (.wav), demonstrating versatility and applicability in different data domains. To facilitate user interaction, a graphical user interface (GUI) was developed with the GTK library, enabling selection of compression algorithms, and input files. Furthermore, to conduct a rigorous comparative analysis, Python scripts were employed to process experimental data and generate graphical representations of key performance metrics such as compression ratio and execution time for different file types. The experimental results indicate that no single algorithm universally outperforms others; rather, each one exhibits specific advantages depending on the file type and data characteristics. This highlights the importance of choosing the appropriate compression method tailored to the application's needs. The project contributes as a practical tool for data compression and an educational resource that aids in understanding fundamental compression algorithms and their behavior in real-world scenarios.

Keywords: Data compression, compression algorithms, Huffman coding, LZ77, LZW, GZIP, C++, Python, GTK.

Lista de ilustrações

Figura 1	— Entropia de uma moeda em função de p	20
Figura 2	— Exemplo de uma Árvore de Huffman	26
Figura 3	— Header do arquivo BMP	34
Figura 4	— Header do arquivo WAV	35

Lista de tabelas

Lista de abreviaturas e siglas

LZ77	Lempel-Ziv 1977 (algoritmo de compressão)
LZW	Lempel-Ziv-Welch (algoritmo de compressão)
GZIP	GNU zip (algoritmo de compressão)
GTK	GIMP Toolkit (biblioteca gráfica para interface)

Sumário

1	INTRODUÇÃO	12
1.1	Problema	13
1.2	Objetivos	13
1.2.1	Objetivo geral	13
1.2.2	Objetivos específicos	14
2	FUNDAMENTAÇÃO TEÓRICA	15
2.1	Teoria da Informação	15
2.1.1	Quantificação da Informação de Hartley	15
2.1.2	Conteúdo informacional de Shannon	18
2.1.3	Entropia	19
2.1.4	Redundância	20
2.1.5	Desigualdade de Kraft-McMillan e taxa de compressão	22
2.2	Compressão de dados	23
2.3	Algoritmos clássicos de compressão sem perdas	24
2.3.1	Codificação de Huffman	25
2.3.1.1	Funcionamento do algoritmo	25
2.3.1.2	Exemplo prático	25
2.3.1.3	Propriedades matemáticas	26
2.3.1.4	Variações do algoritmo	27
2.3.2	LZ77	28
2.3.2.1	Funcionamento do algoritmo	28
2.3.2.2	Exemplo prático	29
2.3.2.3	Variações e otimizações	29
2.3.2.4	Análise de complexidade	29
2.3.2.5	Propriedades teóricas	30
2.3.2.6	Parâmetros de configuração	30
2.3.3	LZW	31
2.3.4	GZIP	32
3	IMPLEMENTAÇÃO	33
3.0.1	Manipulação de Headers BMP e WAV	33
3.0.1.1	Implementação da Classe BMP	35
3.0.1.2	Implementação da Classe WAV	38
3.0.2	Classe BitStream	39
3.0.3	Huffman	39

3.0.3.1	Huffman TXT	39
3.0.3.2	Huffman BMP	39
3.0.3.3	Huffman WAV	39
3.0.4	LZ77	39
3.0.5	LZW	39
3.0.6	GZIP	39
4	ANÁLISE COMPARATIVA	40
5	CONCLUSÃO	41
	REFERÊNCIAS	42

1 Introdução

A compressão de dados é uma técnica essencial na ciência da computação, utilizada para reduzir o tamanho dos arquivos, otimizando o uso de recursos e acelerando a transmissão de dados – aspectos cruciais na era digital em que vivemos.

O desenvolvimento da área teve início na década de 1950, com os primeiros métodos voltados à otimização do espaço de armazenamento. Entre estes métodos, destaca-se o algoritmo de Huffman (HUFFMAN, 1952), que emprega uma codificação de prefixo variável, atribuindo códigos menores aos símbolos mais frequentes e maiores aos menos frequentes. Essa abordagem se consolidou como um dos pilares da área e é utilizada em formatos como ZIP e GZIP.

Posteriormente, novos algoritmos surgiram. Um dos mais significativos é o LZ77, proposto por Abraham Lempel e Jacob Ziv (ZIV; LEMPEL, 1977), que adota uma estratégia baseada em dicionários, substituindo sequências repetidas por referências a posições anteriores. A partir desta ideia, o LZW vem à tona aprimorando o LZ77 ao construir dinamicamente um dicionário de *strings* durante a compressão (WELCH, 1984), sendo amplamente utilizado em formatos como GIF e TIFF.

Outro algoritmo relevante é o GZIP, amplamente adotado em sistemas Unix e aplicações web. Ele combina a eficiência da codificação de Huffman com o LZ77, alcançando boas taxas de compressão sem perdas. Sua popularidade decorre da combinação de alta eficácia com a facilidade de descompressão.

Apesar do surgimento de novas técnicas, os algoritmos de Huffman, LZ77, LZW e GZIP continuam a formar a espinha dorsal dos métodos clássicos de compressão de dados e são amplamente utilizados em diversas aplicações.

Nos últimos anos, surgiram algoritmos mais modernos, como o Brotli – desenvolvido pela Google – que combina codificação de Huffman e transformações de fluxo de dados para obter taxas de compressão superiores às do GZIP (ALAKUIJALA et al., 2016). Outro exemplo é o Zstandard (Zstd), criado pelo Facebook, que alia velocidade e compressão eficiente, sendo adequado tanto para compressão em tempo real quanto para grandes volumes de dados (COLLET, 2016).

Além disto, algoritmos baseados em aprendizado de máquina têm ganhado destaque, principalmente na compressão de imagens e áudio. Redes neurais como GANs (para imagens) e modelos como o WaveNet (para áudio) (DeepMind, 2016) vêm sendo explorados para melhorar a qualidade das compressões. No campo da compressão de vídeo, codecs como HEVC (H.265) e VVC vêm sendo desenvolvidos

para oferecer compressões mais eficazes em vídeos de alta definição.

A compressão de dados desempenha, portanto, um papel estratégico em diversas áreas tecnológicas, impactando diretamente sistemas de armazenamento, transmissão de dados, streaming, redes e computação em nuvem. A análise de desempenho dos algoritmos, considerando métricas como taxa de compressão e tempo de execução, é essencial para a escolha do método mais adequado a cada situação.

Neste contexto, apresenta-se um estudo aprofundado sobre os algoritmos clássicos de compressão Huffman, LZ77, LZW e GZIP. São abordados os fundamentos teóricos de cada método, suas implementações práticas em linguagem C++ com suporte a arquivos nos formatos .txt, .bmp e .wav, e uma interface gráfica desenvolvida com GTK. A avaliação experimental é realizada por meio de comparações de desempenho, com auxílio de scripts em Python para geração de gráficos e análise dos resultados. O objetivo é compreender o comportamento dos algoritmos em diferentes tipos de dados, destacando suas vantagens, limitações e aplicações mais adequadas.

O restante deste trabalho está organizado da seguinte forma: o Capítulo 2 apresenta os fundamentos teóricos sobre compressão de dados, abordando a Teoria da Informação e os algoritmos Huffman, LZ77, LZW e GZIP. O Capítulo 3 descreve a implementação dos algoritmos, incluindo aspectos técnicos e ferramentas utilizadas. No Capítulo 4, é realizada uma análise comparativa com base nos testes realizados, considerando taxa de compressão e tempo de execução. Por fim, o Capítulo 5 apresenta as conclusões do trabalho e sugestões para pesquisas futuras.

1.1 Problema

Como os diferentes algoritmos clássicos de compressão de dados (Huffman, LZ77, LZW e GZIP) se comparam em termos de eficiência de compressão e tempo de execução, e qual o impacto dessas métricas em aplicações práticas que lidam com texto, imagem e áudio?

1.2 Objetivos

1.2.1 Objetivo geral

Investigar e comparar o desempenho de algoritmos clássicos de compressão de dados sem perdas, a saber: Huffman, LZ77, LZW e GZIP, por meio da implementação prática dessas técnicas, aplicando-as a diferentes tipos de arquivos e analisando sua eficiência em termos da taxa de compressão e tempo de execução.

1.2.2 Objetivos específicos

- Estudar os fundamentos teóricos dos algoritmos de compressão sem perdas selecionados, compreendendo o seu funcionamento, estrutura e aplicação histórica.
- Implementar, em linguagem C++, os algoritmos Huffman, LZ77, LZW e GZIP, com foco na compressão e descompressão de arquivos de texto (.txt), imagem bitmap (.bmp) e áudio PCM (.wav).
- Desenvolver uma interface gráfica com a biblioteca GTK, que permita ao usuário escolher o algoritmo e o algoritmo de entrada de forma interativa.
- Criar scripts em Python para a análise comparativa, organizando e visualizando os resultados experimentais por meio de gráficos e tabelas.
- Avaliar os pontos fortes e fracos de cada algoritmo, considerando o tipo de dado, a estrutura do arquivo, o tempo de processamento e a taxa de compressão.
- Identificar o algoritmo mais adequado a cada cenário específico, com base nos resultados experimentais obtidos.

2 Fundamentação teórica

Este capítulo apresenta os fundamentos teóricos que embasam o desenvolvimento deste trabalho. Para compreender o funcionamento e as diferenças entre os algoritmos de compressão de dados abordados, é necessário introduzir alguns conceitos essenciais.

Primeiramente, será apresentada a teoria da informação, a qual fornece o embasamento matemático para a compressão de dados sem perdas, por meio de conceitos como entropia e redundância. Em seguida, serão discutidos os princípios gerais da compressão de dados, com foco nas técnicas sem perdas, suas aplicações e métricas de desempenho. Por fim, serão detalhados os algoritmos clássicos de compressão sem perdas utilizados neste trabalho: Huffman, LZ77, LZW e GZIP, incluindo suas características, funcionamento e aplicações práticas.

A estrutura deste capítulo está organizada da seguinte forma:

- **Seção 2.1** – Apresenta os principais conceitos da Teoria da Informação, com ênfase na entropia de Shannon e seu papel na compressão de dados;
- **Seção 2.2** – Discute os fundamentos da compressão de dados sem perdas, destacando seus objetivos, métricas e distinções em relação à compressão com perdas;
- **Seção 2.3** – Descreve detalhadamente os algoritmos Huffman, LZ77, LZW e GZIP, explicando seus princípios de funcionamento, vantagens, limitações e casos de uso.

2.1 Teoria da Informação

A Teoria da Informação, proposta por Claude Shannon em 1948, constitui o alicerce teórico para diversos processos de codificação e compressão de dados na ciência da computação e engenharia de comunicações. Seu objetivo central é quantificar a informação contida em uma mensagem e estabelecer limites teóricos para a eficiência da transmissão e armazenamento de dados (SHANNON, 1948).

2.1.1 Quantificação da Informação de Hartley

Em 1928, Ralph V. L. Hartley publicou o artigo *Transmission of Information* (HARTLEY, 1928), no qual propôs uma medida quantitativa para a informação baseada

unicamente em aspectos físicos e objetivos do processo de comunicação. Sua motivação era eliminar fatores subjetivos, como interpretação ou significado, e estabelecer uma definição útil para aplicações em engenharia.

Hartley introduz seu raciocínio a partir da observação de sistemas de comunicação reais, como o sistema de cabo telegráfico submarino operado manualmente. Nesse sistema, o operador utiliza uma chave com três posições, por exemplo, tensão positiva, tensão nula e tensão negativa, cada uma representando um símbolo físico distinto. A mensagem enviada é uma sequência dessas seleções, e o receptor interpreta os sinais registrados por um oscilógrafo em uma fita fotosensível.

O ponto fundamental é que, em cada instante de envio, o operador faz uma escolha consciente entre múltiplos símbolos possíveis. À medida que as seleções ocorrem, mais possibilidades de mensagens são descartadas, o que equivale a dizer que a informação vai se tornando mais específica.

Hartley argumenta que a quantidade de informação transmitida deve depender do número de símbolos disponíveis a cada escolha e do número total de escolhas feitas. Por exemplo, se há $s = 3$ símbolos possíveis por escolha e n escolhas são feitas, o número total de sequências distintas possíveis é:

$$N = s^n$$

Esse valor representa a quantidade de mensagens distintas que o sistema é capaz de transmitir fisicamente. No entanto, usar diretamente esse número como medida de informação não é prático, pois ele cresce exponencialmente com o número de seleções. Para obter uma medida linear e proporcional ao número de escolhas, Hartley propõe tomar o logaritmo desse valor:

$$H = \log_b N = \log_b(s^n) = n \log_b s$$

Essa expressão garante que:

- A medida de informação seja proporcional ao número de escolhas n ;
- Duas mensagens independentes tenham informação total igual à soma das informações individuais (propriedade de aditividade);
- O valor seja expresso em uma unidade prática — como bits, ao usar base $b = 2$.

Exemplos:

- **Exemplo 0:** Considere um sistema em que cada seleção permite escolher entre três símbolos distintos (por exemplo, $s = 3$). Se forem feitas 5 seleções ($n = 5$), o número total de mensagens possíveis é:

$$N = 3^5 = 243$$

A quantidade de informação total transmitida será:

$$H = \log_2 243 \approx 7,925 \text{ bits}$$

Ou, usando a fórmula direta:

$$H = 5 \log_2 3 \approx 5 \times 1,585 \approx 7,925 \text{ bits}$$

Isso significa que, em média, cada seleção transmite aproximadamente 1,585 bits de informação.

- **Exemplo 1:** Suponha um sistema onde o transmissor pode escolher entre 2 símbolos diferentes (como 0 e 1, em um código binário). Uma única escolha transmite:

$$I = \log_2 2 = 1 \text{ bit}$$

Com 8 escolhas, tem-se:

$$H = 8 \log_2 2 = 8 \text{ bits}$$

- **Exemplo 2:** Considere agora um sistema com 4 símbolos distintos (por exemplo, A, B, C, D). Cada escolha transmite:

$$I = \log_2 4 = 2 \text{ bits}$$

Após 5 seleções, a quantidade total de informação transmitida é:

$$H = 5 \log_2 4 = 10 \text{ bits}$$

- **Exemplo 3:** Em um sistema decimal com 10 símbolos (de 0 a 9), uma única seleção transmite:

$$I = \log_2 10 \approx 3,32 \text{ bits}$$

Três seleções transmitiriam aproximadamente:

$$H = 3 \log_2 10 \approx 9,97 \text{ bits}$$

Esses exemplos ilustram que, quanto maior o número de símbolos possíveis por escolha, maior é o conteúdo de informação por escolha individual. A grande sacada de Hartley foi perceber que a informação é proporcional ao número de seleções, e que o logaritmo do número de possibilidades elimina a explosão exponencial, fornecendo uma medida prática e linear.

2.1.2 Conteúdo informacional de Shannon

A abordagem de Hartley, no entanto, assume que todos os símbolos têm igual probabilidade de ocorrência, o que nem sempre é verdadeiro na prática. Para resolver essa limitação, Shannon generalizou o conceito e definiu o conteúdo informacional¹ associada a um evento x com probabilidade $P(x)$:

$$I(x) = -\log_b P(x)$$

A ideia principal é que o valor informacional de uma mensagem depende do grau em que a mesma é surpreendente. Se um evento altamente provável ocorrer, a mensagem carrega pouca informação; por outro lado, se um evento improvável acontece, diz-se que carrega muita informação.

Por definição, se $P(x) = 0$ então $I(x) = 0$, ou seja, se o evento for impossível, o valor informacional é nulo.

Considere dois eventos distintos:

- Evento A : "Vai chover amanhã", com probabilidade $P(A) = 0,9$;
- Evento B : "Vai nevar amanhã", com probabilidade $P(B) = 0,01$.

Aplicando a fórmula do conteúdo informacional com base 2:

$$I(A) = -\log_2(0,9) \approx 0,152 \text{ bits}$$

$$I(B) = -\log_2(0,01) \approx 6,644 \text{ bits}$$

Isso significa que a ocorrência de A transmite pouca informação, pois é esperada. Já a ocorrência de B transmite uma quantidade significativamente maior de informação, pois representa um evento altamente improvável — portanto, mais surpreendente.

Esse exemplo mostra que a informação está relacionada com a incerteza: quanto menor a probabilidade de um evento, maior o seu conteúdo informacional.

¹ Em inglês, information content, surprisal ou self-information

2.1.3 Entropia

A entropia de uma fonte, então, é o valor esperado do conteúdo de informação de seus símbolos:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_b P(x_i)$$

Essa formulação considera a estrutura estatística dos dados, permitindo medir a incerteza média da fonte — e, conseqüentemente, o limite teórico mínimo para compressão sem perdas.

Considere uma variável aleatória X que representa o lançamento de uma moeda, onde os possíveis resultados são:

- $x_1 = \text{cara}$, com probabilidade $P(x_1) = p$
- $x_2 = \text{coroa}$, com probabilidade $P(x_2) = 1 - p$

A entropia da variável X é:

$$H(X) = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

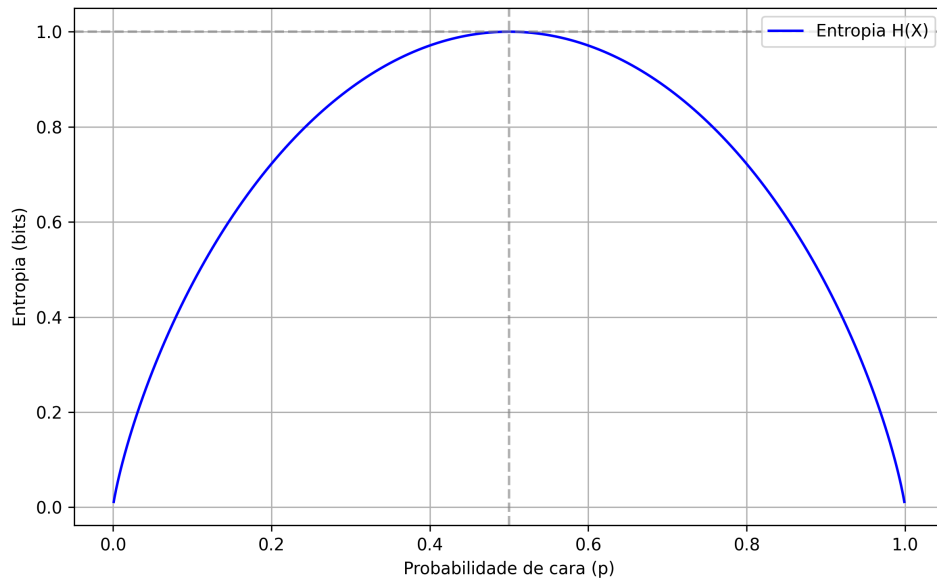
Essa função atinge seu valor máximo quando $p = 0,5$, ou seja, quando a moeda é justa e os dois eventos são igualmente prováveis. Nesse caso:

$$H(X) = -0,5 \log_2 0,5 - 0,5 \log_2 0,5 = -2 \times 0,5 \times (-1) = 1 \text{ bit}$$

Esse valor representa o máximo de incerteza possível para uma variável binária: não se sabe nada sobre qual será o resultado. Por outro lado, se a moeda estiver viciada e sempre cair do mesmo lado (por exemplo, $p = 0$ ou $p = 1$), então:

$$H(X) = -1 \cdot \log_2 1 = 0 \text{ bits}$$

Ou seja, não há incerteza — o resultado é completamente previsível, e, portanto, não há informação nova sendo transmitida.

Figura 1 – Entropia de uma moeda em função de p 

Fonte: Elaborada pelo autor

2.1.4 Redundância

A redundância representa a diferença entre o comprimento médio real de codificação e a entropia da fonte. Em outras palavras, trata-se da quantidade de informação excedente que pode estar presente nos dados, muitas vezes introduzida por razões como segurança, integridade ou simplicidade de codificação. Quanto maior for essa redundância estatística, maior será o potencial de compressão sem perdas.

A relação entre a entropia $H(X)$ de uma fonte e o comprimento médio L de um código pode ser expressa por:

$$R = L - H(X)$$

onde R é a redundância. Um código é considerado eficiente quando sua média de bits por símbolo se aproxima da entropia da fonte, ou seja, quando $R \rightarrow 0$.

O principal objetivo dos algoritmos de compressão de dados é justamente reduzir essa redundância presente na representação original da informação. Ao identificar e eliminar padrões repetitivos, estruturas previsíveis ou codificações desnecessárias, é possível representar os dados com uma quantidade menor de bits, mantendo integralmente a informação. Assim, a compressão atua aproximando o código do limite teórico mínimo imposto pela entropia.

Considere uma fonte com três símbolos e as seguintes frequências:

Símbolo	Frequência	Probabilidade
A	50	0,5
B	30	0,3
C	20	0,2

A entropia dessa fonte, em bits por símbolo, é:

$$H(X) = -(0,5 \log_2 0,5 + 0,3 \log_2 0,3 + 0,2 \log_2 0,2) \approx 1,485 \text{ bits/símbolo}$$

Após aplicar um algoritmo de compressão de dados qualquer, suponha que os símbolos sejam codificados da seguinte forma:

- A $\rightarrow$ 1 bit
- B $\rightarrow$ 2 bits
- C $\rightarrow$ 2 bits

O comprimento médio da codificação é:

$$L = (0,5 \times 1) + (0,3 \times 2) + (0,2 \times 2) = 1,5 \text{ bits/símbolo}$$

Portanto, a redundância é:

$$R = L - H(X) = 1,5 - 1,485 = 0,015 \text{ bits/símbolo}$$

Esse pequeno valor de R indica que o código comprimido se aproxima do limite teórico, demonstrando alta eficiência.

É importante destacar que a entropia $H(X)$ é uma propriedade intrínseca da fonte, determinada unicamente pela distribuição de probabilidades dos símbolos, e representa a quantidade média de informação por símbolo. Portanto, seu valor não se altera durante o processo de compressão. O que se modifica é o comprimento médio de codificação L , que tende a se aproximar de $H(X)$ à medida que o código se torna mais eficiente.

Dessa forma, ao reduzir o número médio de bits por símbolo, não ocorre uma diminuição da entropia da fonte, mas sim um aumento da quantidade de informação útil carregada por cada bit. Esse efeito pode ser expresso pela razão entre a entropia e o comprimento médio:

$$\eta = \frac{H(X)}{L}$$

onde η representa a eficiência do código. À medida que $L \rightarrow H(X)$, tem-se $\eta \rightarrow 1$, indicando que cada bit transmitido se torna mais informativo e menos redundante. Esse comportamento evidencia a eficácia da compressão, que reduz o volume de dados a serem armazenados ou transmitidos sem comprometer o conteúdo informacional original.

2.1.5 Desigualdade de Kraft-McMillan e taxa de compressão

Ao estudar a relação entre entropia e os algoritmos de compressão, dois conceitos adicionais merecem destaque: a desigualdade de Kraft-McMillan e a taxa de compressão.

A desigualdade de Kraft-McMillan estabelece uma condição necessária e suficiente para a existência de códigos prefixos, ou seja, códigos em que nenhum código é prefixo de outro, como no algoritmo de Huffman. Seja l_i o comprimento do código associado ao símbolo i . A desigualdade é dada por:

$$\sum_{i=1}^n 2^{-l_i} \leq 1 \quad (2.1)$$

O significado da desigualdade é que, se os comprimentos l_i satisfazem a condição, então é matematicamente garantido que existe ao menos um conjunto de códigos de prefixo que pode ser atribuído aos símbolos. Em outras palavras, é possível construir um código em que nenhum símbolo seja prefixo de outro, assegurando uma decodificação instantânea e sem ambiguidades. A desigualdade, portanto, não descreve o código em si, mas estabelece a condição necessária e suficiente para a sua existência.

Por exemplo, considere um conjunto de símbolos $\{A, B, C\}$ com comprimentos de código $l_A = 1$, $l_B = 2$ e $l_C = 2$. Temos:

$$2^{-1} + 2^{-2} + 2^{-2} = 0,5 + 0,25 + 0,25 = 1$$

Como a desigualdade é satisfeita, é possível construir um código de prefixo válido, como $A = 0$, $B = 10$ e $C = 11$.

Ao se tratar da eficiência de um algoritmo, define-se a *taxa de compressão* como a razão entre o tamanho do arquivo original e o tamanho do arquivo comprimido:

$$\text{Taxa de compressão} = \frac{T_{\text{original}}}{T_{\text{comprimido}}} \quad (2.2)$$

Além disso, é comum expressar a *redução percentual* obtida pela compressão:

$$\text{Redução} = \left(1 - \frac{T_{\text{comprimido}}}{T_{\text{original}}} \right) \times 100\% \quad (2.3)$$

Essas métricas permitem comparar algoritmos de compressão distintos de forma objetiva, complementando a análise teórica baseada em entropia e redundância.

Por fim, cabe destacar que a entropia pode ser definida não apenas em termos de símbolos isolados, mas também considerando dependências entre símbolos consecutivos, conhecida como *entropia de ordem superior*. Esse conceito é explorado por algoritmos como LZ77 e LZW, que buscam padrões e repetições em sequências, indo além da simples análise da frequência de símbolos individuais.

2.2 Compressão de dados

A compressão de dados consiste em um conjunto de técnicas destinadas a representar informações de forma mais compacta, reduzindo o espaço necessário para armazenamento ou o tempo exigido para transmissão. Trata-se de um campo fundamental da ciência da computação, com aplicações diretas em praticamente todas as áreas que envolvem o tratamento de dados digitais, como bancos de dados, sistemas de arquivos, comunicação em rede, multimídia e computação em nuvem.

O princípio básico da compressão é a exploração da redundância presente nos dados. Arquivos digitais, independentemente de sua natureza (texto, imagem, áudio ou vídeo), frequentemente apresentam padrões estatísticos, repetições ou estruturas previsíveis que podem ser representados de maneira mais eficiente. A eliminação ou redução dessa redundância resulta em representações mais curtas, possibilitando economia de recursos sem comprometer a utilidade da informação.

De maneira geral, os métodos de compressão podem ser classificados em duas categorias: compressão sem perdas e compressão com perdas.

Na compressão sem perdas, os dados comprimidos podem ser restaurados integralmente em sua forma original após a descompressão. Não ocorre perda de informação, sendo essa abordagem indispensável em contextos em que a integridade dos dados é crucial, como em documentos de texto, programas executáveis, imagens médicas e determinados formatos de áudio.

Já a compressão com perdas permite alcançar taxas de compressão mais elevadas ao descartar parte das informações, geralmente aquelas que são perceptualmente redundantes ou irrelevantes. Embora o arquivo reconstruído não seja idêntico ao original, a diferença pode ser imperceptível ao usuário final. Esse modelo é amplamente

utilizado em mídias digitais, como fotografias (JPEG), música (MP3, AAC) e vídeo (MPEG, H.264).

O presente trabalho concentra-se exclusivamente na compressão sem perdas, uma vez que seu objetivo é a análise de algoritmos clássicos de codificação que buscam aproximar-se do limite teórico estabelecido pela entropia da fonte de informação. Essa escolha se justifica não apenas pela relevância prática da compressão sem perdas em diversos domínios, mas também por seu caráter teórico, que permite avaliar a eficiência dos algoritmos em termos formais, sem a interferência de fatores perceptuais.

Para avaliar a eficácia de um algoritmo de compressão, algumas métricas são comumente empregadas:

- Taxa de compressão: relação entre o tamanho do arquivo comprimido e o tamanho do arquivo original, expressa geralmente em porcentagem;
- Ganho de compressão: medida complementar, definida pela razão entre o tamanho original e o tamanho comprimido, indicando quantas vezes o volume de dados foi reduzido;
- Tempo de execução: tempo necessário para realizar os processos de compressão e descompressão;

Em síntese, a compressão de dados busca otimizar recursos computacionais ao reduzir a quantidade de informação redundante em uma representação digital. A decisão entre compressão sem perdas ou com perdas depende do contexto e das exigências de cada aplicação. No âmbito deste trabalho, o foco em algoritmos de compressão sem perdas permite investigar de forma detalhada a relação entre teoria da informação e implementação prática, bem como comparar a eficiência de diferentes abordagens clássicas nesse domínio.

2.3 Algoritmos clássicos de compressão sem perdas

Os algoritmos de compressão sem perdas constituem a base teórica e prática de diversos sistemas computacionais modernos. Entre os mais conhecidos e aplicados estão os algoritmos de Huffman, LZ77, LZW e GZIP, cada um com características específicas que os tornam adequados a diferentes contextos de uso. A seguir, apresentam-se seus princípios de funcionamento, vantagens, limitações e principais aplicações.

2.3.1 Codificação de Huffman

O algoritmo de Huffman, proposto por David A. Huffman em 1952 (HUFFMAN, 1952), é um método de codificação estatística que constrói códigos de prefixo ótimos para compressão sem perdas. O algoritmo fundamenta-se no princípio de atribuir códigos binários de comprimentos variáveis aos símbolos de uma fonte, de modo que símbolos mais frequentes recebem códigos mais curtos, minimizando assim o comprimento médio da codificação.

2.3.1.1 Funcionamento do algoritmo

A construção do código de Huffman é realizada através da criação de uma árvore binária, seguindo os seguintes passos:

1. **Análise de frequência:** Conta-se a frequência de ocorrência de cada símbolo no conjunto de dados a ser comprimido;
2. **Criação de nós folha:** Cada símbolo é representado por um nó folha, associado à sua respectiva frequência;
3. **Construção da árvore:** Iterativamente, os dois nós com menores frequências são combinados em um novo nó pai, cuja frequência é a soma das frequências dos nós filhos. Este processo continua até restar apenas um nó (a raiz);
4. **Atribuição de códigos:** Percorre-se a árvore da raiz até as folhas, atribuindo '0' para ramos esquerdos e '1' para ramos direitos (ou vice-versa). O código de cada símbolo é formado pela sequência de bits do caminho da raiz até sua folha correspondente.

2.3.1.2 Exemplo prático

Considere uma mensagem com os símbolos e suas respectivas frequências:

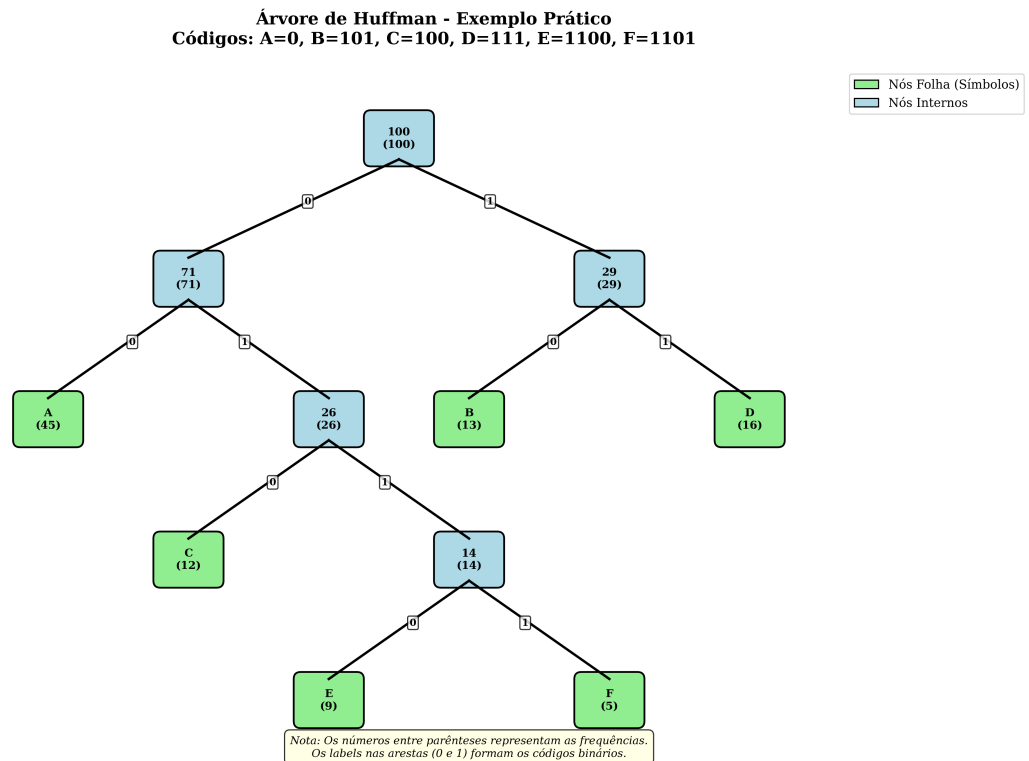
Símbolo	Frequência	Probabilidade	Código Huffman
A	45	0,45	0
B	13	0,13	101
C	12	0,12	100
D	16	0,16	111
E	9	0,09	1100
F	5	0,05	1101

O comprimento médio da codificação seria:

$$L = 0,45 \times 1 + 0,13 \times 3 + 0,12 \times 3 + 0,16 \times 3 + 0,09 \times 4 + 0,05 \times 4 = 2,04 \text{ bits/símbolo}$$

Comparado com uma codificação binária fixa (que exigiria $\lceil \log_2 6 \rceil = 3$ bits por símbolo), o algoritmo de Huffman oferece uma redução significativa no tamanho da representação.

Figura 2 – Exemplo de uma Árvore de Huffman



Fonte: Elaborada pelo autor

2.3.1.3 Propriedades matemáticas

O algoritmo de Huffman possui propriedades teóricas importantes:

- **Otimidade:** Para uma fonte com probabilidades conhecidas e símbolos independentes, o código de Huffman minimiza o comprimento médio de codificação entre todos os códigos de prefixo possíveis;
- **Satisfação da desigualdade de Kraft:** Os comprimentos dos códigos gerados sempre satisfazem $\sum_{i=1}^n 2^{-l_i} = 1$, garantindo a existência de um código de prefixo válido;
- **Limitação pela entropia:** O comprimento médio L do código de Huffman satisfaz $H(X) \leq L < H(X) + 1$, onde $H(X)$ é a entropia da fonte.

2.3.1.4 Variações do algoritmo

Existem algumas variações importantes do algoritmo básico de Huffman:

- **Huffman adaptativo:** Ajusta dinamicamente o código conforme novos símbolos são processados, sem necessidade de conhecimento prévio das frequências;
- **Huffman canônico:** Produz uma representação padronizada do código que facilita a transmissão da tabela de códigos;
- **Huffman de duas passadas:** Realiza uma primeira passada para calcular frequências e uma segunda para codificar, sendo mais eficiente em termos de memória.

Vantagens:

- Algoritmo ótimo para fontes com probabilidades conhecidas e símbolos independentes;
- Simplicidade de implementação e baixo overhead computacional;
- Garantia matemática de código de prefixo válido;
- Eficiência comprovada em dados com distribuições não uniformes.

Limitações:

- Eficiência dependente da precisão das estimativas de probabilidade dos símbolos;
- Não se adapta dinamicamente a variações estatísticas da fonte (versão estática);
- Desempenho subótimo em fontes com distribuições uniformes;
- Necessidade de transmitir ou armazenar a tabela de códigos junto com os dados comprimidos.

Casos de uso:

- Componente fundamental em formatos de compressão como DEFLATE (ZIP, PNG);
- Codificação de coeficientes em compressão de imagem (JPEG);
- Protocolos de comunicação que requerem compressão em tempo real;
- Sistemas de armazenamento que priorizam alta taxa de compressão para dados textuais.

2.3.2 LZ77

O algoritmo LZ77, proposto por Abraham Lempel e Jacob Ziv em 1977 (ZIV; LEMPEL, 1977), representa um marco na compressão de dados sem perdas ao introduzir o conceito de compressão baseada em dicionário dinâmico. Diferentemente dos métodos estatísticos como Huffman, que dependem da frequência de símbolos individuais, o LZ77 explora a redundância temporal presente em sequências de dados, identificando e codificando repetições de subsequências já observadas.

2.3.2.1 Funcionamento do algoritmo

O LZ77 opera com base em uma janela deslizante que divide o fluxo de entrada em duas regiões principais:

- **Buffer de busca:** região que contém os dados já processados, servindo como dicionário para identificar repetições;
- **Buffer de antecipação:** região que contém os dados ainda não processados, onde se busca por correspondências com o buffer de busca.

O algoritmo funciona através dos seguintes passos:

1. **Inicialização:** Define-se o tamanho do buffer de busca tipicamente entre 4KB e 32KB e do buffer de antecipação entre 4 e 258 bytes;
2. **Busca por correspondências:** Para cada posição no buffer de antecipação, procura-se a maior subsequência que tenha uma correspondência no buffer de busca;
3. **Codificação:** Se uma correspondência é encontrada, ela é codificada como um triplo (distância, comprimento, próximo_símbolo), onde:
 - *distância*: número de posições para trás onde a correspondência foi encontrada;
 - *comprimento*: número de símbolos que correspondem;
 - *próximo_símbolo*: primeiro símbolo após a correspondência que não faz parte dela.
4. **Avanço da janela:** A janela desliza para a direita, incorporando os símbolos processados ao buffer de busca e carregando novos símbolos no buffer de antecipação;
5. **Repetição:** O processo continua até que todos os dados sejam processados.

2.3.2.2 Exemplo prático

Considere a sequência de entrada: "AACCAACABCABAAAC"

Assumindo uma janela de busca de tamanho 6 e buffer de antecipação de tamanho 4:

Passo	Buffer de busca	Buffer antecipação	Saída
1	(vazio)	AACA	(0,0,A)
2	A	ACAA	(1,1,C)
3	AAC	AACA	(3,4,B)
4	CAACAB	CABA	(3,3,A)
5	ABCABA	AAC	(1,2,C)

A saída comprimida seria:

(0, 0, A) ; (1, 1, C) ; (3, 4, B) ; (3, 3, A) ; (1, 2, C)

2.3.2.3 Variações e otimizações

O algoritmo LZ77 possui diversas variações que melhoram seu desempenho:

- **LZSS (Lempel-Ziv-Storer-Szymanski):** Elimina o campo "próximo_símbolo" do triplo, usando um bit de flag para distinguir entre literais e referências, resultando em melhor compressão;
- **LZ77 com hash:** Utiliza tabelas hash para acelerar a busca por correspondências, reduzindo a complexidade de $O(n \cdot w)$ para $O(n)$, onde w é o tamanho da janela;
- **LZ77 com árvores de sufixos:** Emprega estruturas de dados mais sofisticadas para otimizar a busca, especialmente eficaz para textos longos;
- **LZ77 com codificação aritmética:** Combina o LZ77 com codificação aritmética em vez de representação direta dos triplos, melhorando a taxa de compressão.

2.3.2.4 Análise de complexidade

A complexidade computacional do LZ77 depende da implementação:

- **Implementação ingênua:** $O(n \cdot w \cdot l)$, onde n é o tamanho da entrada, w o tamanho da janela de busca e l o comprimento máximo de correspondência;
- **Com otimizações (hash):** $O(n \cdot l)$ no caso médio, $O(n \cdot w \cdot l)$ no pior caso;
- **Espaço:** $O(w + l)$ para armazenar as janelas de busca e antecipação.

A descompressão é significativamente mais rápida, com complexidade $O(n)$, pois requer apenas a expansão sequencial das referências.

2.3.2.5 Propriedades teóricas

O LZ77 possui importantes propriedades teóricas:

- **Universalidade:** É um algoritmo universal, ou seja, para qualquer fonte estacionária e ergódica, a taxa de compressão converge para a taxa de entropia da fonte quando o tamanho da janela tende ao infinito;
- **Adaptabilidade:** Não requer conhecimento prévio das estatísticas da fonte, adaptando-se dinamicamente aos padrões presentes nos dados;
- **Garantia de não expansão:** No pior caso, o tamanho da saída é limitado pelo tamanho da entrada mais um overhead constante por bloco processado.

2.3.2.6 Parâmetros de configuração

A eficiência do LZ77 depende criticamente da escolha de seus parâmetros:

- **Tamanho do buffer de busca:** Janelas maiores permitem encontrar correspondências mais distantes, mas aumentam o tempo de processamento e o espaço necessário para codificar as distâncias;
- **Comprimento máximo de correspondência:** Valores maiores permitem codificar repetições mais longas, mas requerem mais bits para representar o comprimento;
- **Comprimento mínimo de correspondência:** Correspondências muito curtas podem não compensar o overhead da codificação, sendo mais eficiente tratá-las como literais.

Vantagens:

- Algoritmo universal que não requer conhecimento prévio das estatísticas da fonte;
- Adapta-se dinamicamente aos padrões presentes nos dados;
- Excelente desempenho em dados com repetições estruturais (textos, código fonte, dados binários com padrões);
- Descompressão rápida e de baixa complexidade;
- Base teórica sólida com garantias de convergência para a taxa de entropia.

Limitações:

- Eficiência limitada pelo tamanho da janela de busca, que determina quão distantes podem estar as repetições;
- Desempenho inferior em dados altamente aleatórios ou sem padrões repetitivos;
- Overhead significativo para representar as referências quando as correspondências são curtas;
- Complexidade computacional elevada na compressão, especialmente sem otimizações;
- Sensibilidade à escolha dos parâmetros (tamanhos de janela e buffer).

Casos de uso:

- Base fundamental do algoritmo DEFLATE, usado em formatos ZIP, PNG e HTTP/HTTPS (gzip);
- Compressão de arquivos de texto, código fonte e documentos estruturados;
- Sistemas de backup e arquivamento que priorizam alta taxa de compressão;
- Protocolos de comunicação que requerem compressão em tempo real com boa adaptabilidade;
- Compressão de dados binários com padrões repetitivos (executáveis, bibliotecas).

2.3.3 LZW

O algoritmo LZW (Lempel-Ziv-Welch), desenvolvido por Terry Welch em 1984, é uma variação do LZ78 que constrói dinamicamente um dicionário de padrões encontrados nos dados. Ao contrário do LZ77, que usa referências de deslocamento, o LZW substitui sequências por índices de um dicionário em crescimento contínuo.

Vantagens: simplicidade de implementação, bom desempenho em dados com repetições estruturadas e independência de modelos estatísticos prévios.

Limitações: pode gerar tabelas de dicionário grandes, o que demanda controle para evitar estouro de memória. Em fluxos curtos ou pouco redundantes, a eficiência pode ser reduzida.

Casos de uso: foi amplamente empregado no formato GIF e em implementações de compressão em softwares de sistemas UNIX (como o utilitário `compress`).

2.3.4 GZIP

O GZIP é um formato de compressão que combina o algoritmo LZ77 com a codificação de Huffman, de forma a explorar tanto redundâncias estruturais quanto estatísticas. O fluxo de compressão inicia com a detecção de padrões repetidos pelo LZ77, que são em seguida codificados de maneira mais eficiente através da codificação de Huffman.

Vantagens: alto grau de compressão em uma ampla variedade de dados, eficiência prática comprovada, suporte a múltiplos arquivos e metadados de integridade (CRC).

Limitações: maior complexidade computacional em comparação a algoritmos isolados; embora eficiente, pode não atingir a compressão máxima em todos os cenários.

Casos de uso: amplamente utilizado em sistemas operacionais e protocolos de rede, sendo padrão em distribuições de arquivos no ambiente UNIX/Linux e em transmissões web (HTTP/HTTPS).

3 Implementação

Este capítulo apresenta a implementação dos principais componentes desenvolvidos no projeto, abordando desde a manipulação dos headers dos formatos BMP e WAV até a construção das classes e algoritmos responsáveis pela compressão e descompressão de dados. São detalhados os módulos referentes à leitura e interpretação de arquivos, a classe auxiliar *BitStream* para manipulação de bits, e os algoritmos de compressão Huffman, LZ77, LZW e GZIP, explicando suas estruturas, funcionamento e integração com a API desenvolvida.

3.0.1 Manipulação de Headers BMP e WAV

Durante o desenvolvimento deste trabalho, foi necessário lidar com diferentes tipos de arquivos como entrada para os algoritmos de compressão e descompressão. Um ponto importante a ser considerado é a estrutura interna desses arquivos, especialmente no que diz respeito à presença ou ausência de cabeçalhos (headers).

Arquivos de texto puro, com extensão `.txt`, são compostos exclusivamente por dados representados em formato ASCII ou Unicode e, portanto, não possuem um cabeçalho estruturado. Esses arquivos não contêm metadados incorporados sobre seu conteúdo, formato ou estrutura, o que simplifica o processo de compressão, já que todos os dados podem ser processados diretamente.

Entretanto, o mesmo não ocorre com arquivos dos formatos `.bmp` (bitmap image) e `.wav` (waveform audio file format). Esses formatos binários possuem estruturas de cabeçalho bem definidas que armazenam metadados essenciais para a correta interpretação dos dados contidos no arquivo. No caso de arquivos BMP, o cabeçalho armazena informações como o tamanho da imagem, profundidade de cor, largura, altura, tipo de compressão (se houver), entre outras. Já no caso dos arquivos WAV, o cabeçalho especifica dados como a taxa de amostragem, número de canais, taxa de bits, tamanho do arquivo, e o formato do áudio (PCM, por exemplo).

Durante o processo de compressão implementado neste trabalho, optou-se por não aplicar compressão sobre os dados de cabeçalho desses arquivos. Essa decisão foi tomada por duas razões principais: (i) o cabeçalho geralmente ocupa uma pequena fração do tamanho total do arquivo, e sua compressão traria benefícios marginais; (ii) preservar o cabeçalho intacto facilita o processo de descompressão e garante que o arquivo reconstruído mantenha sua estrutura original, permitindo sua correta abertura por softwares padrão após a descompressão.

Dessa forma, a implementação realiza a leitura e extração do cabeçalho no

momento da compressão e o armazena separadamente do corpo dos dados. Ao descomprimir o arquivo, o cabeçalho original é restaurado no início do arquivo reconstruído, seguido pelos dados descomprimidos. Esse procedimento assegura que os arquivos BMP e WAV mantêm sua integridade e permanecem compatíveis com os leitores padrão desses formatos.

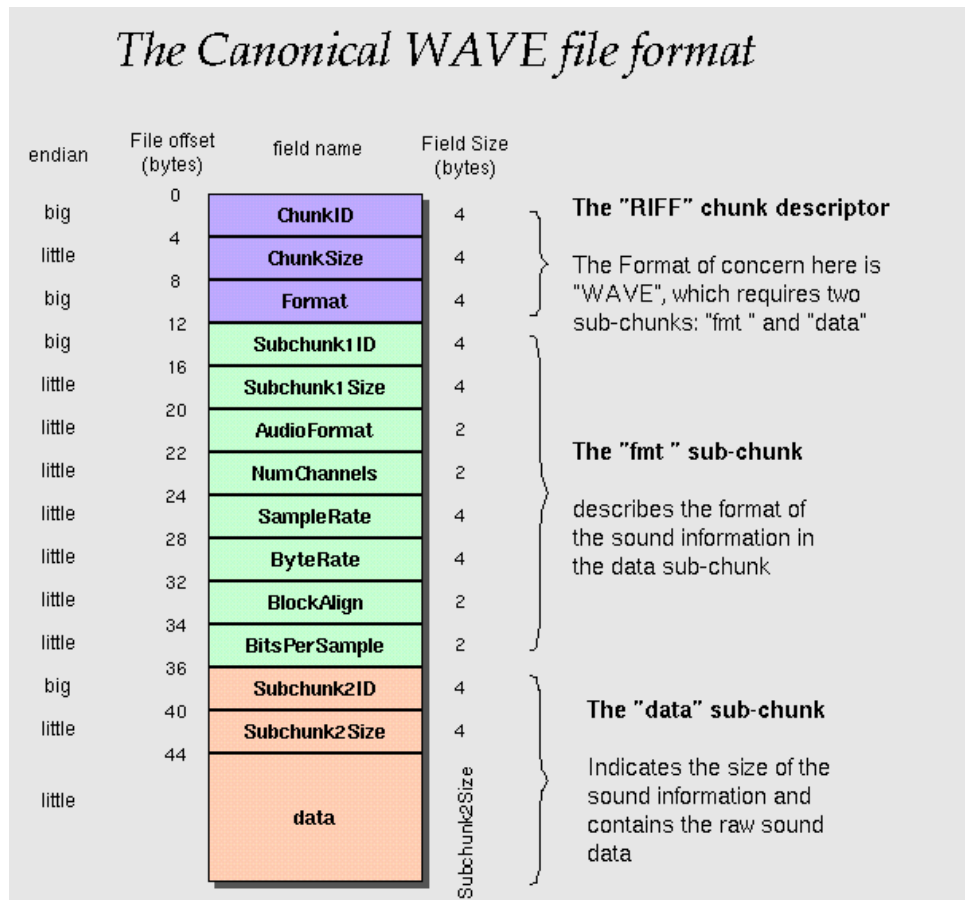
A manipulação correta dos headers foi essencial para garantir a funcionalidade completa do sistema de compressão e descompressão implementado. Sem esse cuidado, os arquivos descomprimidos poderiam se tornar ilegíveis ou corrompidos, uma vez que muitos softwares dependem da presença e exatidão do cabeçalho para interpretar o conteúdo do arquivo de forma apropriada.

Figura 3 – Header do arquivo BMP

Basic BMP File Format		
Name	Size	Description
Header	14 bytes	Windows Structure: BITMAPFILEHEADER
Signature	2 bytes	'BM'
FileSize	4 bytes	File size in bytes
reserved	4 bytes	unused (=0)
DataOffset	4 bytes	File offset to Raster Data
InfoHeader	40 bytes	Windows Structure: BITMAPINFOHEADER
Size	4 bytes	Size of InfoHeader = 40
Width	4 bytes	Bitmap Width
Height	4 bytes	Bitmap Height
Planes	2 bytes	Number of Planes (=1)
BitCount	2 bytes	Bits per Pixel 1 = monochrome palette, NumColors = 1 4 = 4bit palletized, NumColors = 16 8 = 8bit palletized, NumColors = 256 16 = 16bit RGB, NumColors = 65536 (?) 24 = 24bit RGB, NumColors = 16M
Compression	4 bytes	Type of Compression 0 = BI_RGB no compression 1 = BI_RLE8 8bit RLE encoding 2 = BI_RLE4 4bit RLE encoding
ImageSize	4 bytes	(compressed) Size of Image It is valid to set this =0 if Compression = 0
XpixelsPerM	4 bytes	horizontal resolution: Pixels/meter
YpixelsPerM	4 bytes	vertical resolution: Pixels/meter
ColorsUsed	4 bytes	Number of actually used colors
ColorsImportant	4 bytes	Number of important colors 0 = all
ColorTable	4 * NumColors bytes	present only if Info.BitsPerPixel <= 8 colors should be ordered by importance
	Red	1 byte
	Green	1 byte
	Blue	1 byte
	reserved	1 byte
	repeated NumColors times	
Raster Data	Info.ImageSize bytes	The pixel data

Fonte: (Department of Microelectronic Systems, 2025)

Figura 4 – Header do arquivo WAV



Fonte: (SAPP, 2025)

3.0.1.1 Implementação da Classe BMP

Para manipular arquivos BMP, foi implementada uma classe específica que encapsula toda a estrutura do formato, incluindo headers, tabela de cores e dados da imagem. A implementação utiliza estruturas com alinhamento de bytes controlado através da diretiva `#pragma pack` para garantir que os dados sejam lidos corretamente do arquivo binário.

Código 1 – Estrutura da classe BMP

```
#pragma pack(push, 1)
class bmp {
public:
    struct HEADER {
        char signature[2];
        uint32_t file_size;
        uint32_t reserved;
        uint32_t data_offset;
    };
};
```

```

    } header;

    struct INFO_header {
        uint32_t size;
        uint32_t width;
        uint32_t height;
        uint16_t planes;
        uint16_t bits_per_pixel;
        uint32_t compression;
        uint32_t image_size;
        uint32_t x_pixels_per_m;
        uint32_t y_pixels_per_m;
        uint32_t colors_used;
        uint32_t important_colors;
    } info_header;

    uint8_t *color_table;
    uint8_t *data;
    size_t data_size;

    int read(const char* filename);
    void print();
    uint32_t get_color_table_size();
    int has_color_table();
private:
    void read_bmp_data(FILE* file);
};
#pragma pack(pop)

```

A classe BMP implementa métodos essenciais para a leitura e manipulação de arquivos bitmap. O método `read()` é responsável por abrir o arquivo e ler sequencialmente todos os componentes: header principal, info header, tabela de cores (quando aplicável) e dados da imagem. A verificação da existência de tabela de cores é feita através do método `has_color_table()`, que retorna verdadeiro quando o número de bits por pixel é menor ou igual a 8.

O método `read_bmp_data()` implementa uma estratégia de alocação dinâmica de memória que utiliza o padrão de "table doubling" para otimizar o uso de memória durante a leitura dos dados da imagem. Essa abordagem evita a necessidade de conhecer previamente o tamanho dos dados e permite a leitura eficiente de arquivos de

qualquer tamanho.

Código 2 – Método de leitura de dados da imagem BMP

```

size_t buffer_size = 1024;
size_t var_data_size = 0;
uint8_t* var_data = new uint8_t[buffer_size];
if (!var_data) {
    fclose(file);
    EXIT_WITH_ERROR("Memory_allocation_failed");
}
size_t bytes_read;
while((bytes_read = fread(var_data + var_data_size, 1,
                          buffer_size - var_data_size, file)) > 0) {
    var_data_size += bytes_read;
    if(var_data_size >= buffer_size) { // table doubling
        buffer_size *= 2;
        uint8_t* new_data = (uint8_t*)realloc(var_data,
                                                buffer_size);

        if (!new_data) {
            free(var_data);
            fclose(file);
            EXIT_WITH_ERROR("Memory_allocation_failed");
        }
        var_data = new_data;
    }
}
data_size = var_data_size;

data = (uint8_t*)malloc(data_size);
if (!data) {
    free(data);
    fclose(file);
    EXIT_WITH_ERROR("Memory_allocation_failed");
}
memcpy(data, var_data, data_size);
free(var_data);

```

3.0.1.2 Implementação da Classe WAV

Similarmente à classe BMP, a classe WAV foi implementada para manipular arquivos de áudio no formato WAVE PCM. A estrutura segue o padrão RIFF (Resource Interchange File Format), que organiza os dados em chunks identificados por tags de 4 caracteres.

Código 3 – Estrutura da classe WAV

```
#pragma pack(push,1)
class wav {
public:
    struct RIFF_chunk_descriptor {
        char ID[4];
        uint32_t size;
        char format[4];
    } riff_chunk_descriptor;

    struct FMT_subchunk {
        char ID[4];
        uint32_t size;
        uint16_t audio_format;
        uint16_t num_channels;
        uint32_t sample_rate;
        uint32_t byte_rate;
        uint16_t block_align;
        uint16_t bits_per_sample;
    } fmt_subchunk;

    struct DATA_subchunk {
        char ID[4];
        uint32_t size;
    } data_subchunk;

    uint8_t* data;

    int read(std::string filename);
    void print();
};
#pragma pack(pop)
```

A implementação do método de leitura para arquivos WAV segue uma sequên-

cia específica: primeiro lê o descriptor RIFF, depois o subchunk de formato (fmt) e finalmente o subchunk de dados (data). Cada chunk é identificado por sua tag de 4 caracteres, garantindo que a estrutura do arquivo seja respeitada.

Ambas as classes implementam métodos de debug (`print()`) que exibem todas as informações dos headers em formato hexadecimal, facilitando a verificação da integridade dos dados lidos e o desenvolvimento de testes para validar a implementação.

3.0.2 Classe BitStream

3.0.3 Huffman

3.0.3.1 Huffman TXT

3.0.3.2 Huffman BMP

3.0.3.3 Huffman WAV

3.0.4 LZ77

3.0.5 LZW

3.0.6 GZIP

4 Análise comparativa

5 Conclusão

Referências

ALAKUIJALA, J.; SZABADKA, Z.; VANDEVENNE, L.; FARRUGGIA, R.; KLIUCHNIKOV, E.; SZABADKAI, I.; ASSCHE, Z. V. Brotli compressed data format. In: INTERNATIONAL WORLD WIDE WEB CONFERENCES STEERING COMMITTEE. *Proceedings of the 25th International Conference on World Wide Web*. [S.l.], 2016. p. 1593–1603.

COLLET, Y. *Zstandard Compression Algorithm*. 2016. Disponível em: <<https://facebook.github.io/zstd/>>.

DeepMind. *WaveNet: A generative model for raw audio*. 2016. Disponível em: <<https://deepmind.google/research/breakthroughs/wavenet/>>.

Department of Microelectronic Systems. *Structure of BMP file*. 2025. Acesso em: 2025. Disponível em: <http://www.ue.eti.pg.gda.pl/fpgalab/zadania.spartan3/zad_vga_struktura_pliku_bmp_en.html>.

HARTLEY, R. V. L. Transmission of information. *Bell System Technical Journal*, v. 7, n. 3, p. 535–563, 1928.

HUFFMAN, D. A. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, Institute of Radio Engineers, v. 40, n. 9, p. 1098–1101, 1952.

SAPP, C. *WAVE PCM soundfile format*. 2025. Acesso em: 2025. Disponível em: <<http://soundfile.sapp.org/doc/WaveFormat/>>.

SHANNON, C. E. A mathematical theory of communication. *Bell System Technical Journal*, v. 27, n. 3, p. 379–423, 1948.

WELCH, T. A. A technique for high-performance data compression. *Computer*, IEEE, v. 17, n. 6, p. 8–19, 1984.

ZIV, J.; LEMPEL, A. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, IEEE, v. 23, n. 3, p. 337–343, 1977.